Documento tecnico - Segunda entrega

Sistema de Subastas Auct.io

Alumnos: Alejandro Salgado y Lucio Tedeschi

Tecnologias: Android Studio, Java, XML, Node.js, Express y SQL Server

Base de datos: auction

1. Objetivo de la segunda entrega

En esta segunda entrega se presenta una version funcional parcial del sistema de subastas, con backend y frontend conectados entre si. La aplicacion permite que un usuario registrado inicie sesion, consulte subastas disponibles, visualice el catalogo de una subasta, realice una puja, consulte sus medios de pago, vea su historial de participacion, cargue una solicitud de articulo para subastar, consulte su perfil y visualice notificaciones.

El circuito principal integrado implementado es:

```
Login -> Home -> Detalle de subasta -> Catalogo -> Puja -> Historial
```

Este circuito permite demostrar la comunicacion completa entre:

```
Aplicacion Android -> API REST Node.js -> Base de datos SQL Server
```

2. Endpoints API REST implementados

2.1 Test de conexion

Metodo	GET
Endpoint	/api/test
Descripcion	Verifica que la API este funcionando y conectada a SQL Server.
Parametros	No requiere.
Body	No requiere.

Respuesta exitosa:

```
{
  "ok": true,
  "mensaje": "API conectada correctamente",
  "fechaServidor": "2026-05-16T20:00:00.000Z"
}
```

Errores posibles:

Codigo	Descripcion
500	Error interno o problema de conexion con la base de datos.

2.2 Login de usuario

Metodo	POST
Endpoint	/api/auth/login
Descripcion	Permite iniciar sesion con documento y clave.
Parametros	No requiere parametros por URL.
Body	Documento y clave del usuario.

Body:

```
{
  "documento": "30123456",
  "clave": "1234"
}
```

Respuesta exitosa:

```
{
  "mensaje": "Login correcto",
  "usuario": {
    "id": 1,
    "documento": "30123456",
    "nombre": "Alejandro",
    "apellido": "Salgado",
    "email": "alejandro@test.com",
    "estado": "activo",
    "admitido": "si",
    "categoria": "oro"
  }
}
```

Errores posibles:

Codigo	Descripcion
400	Falta documento o clave.
401	Documento o clave incorrecta.
403	Usuario no activo o no admitido.
500	Error interno del servidor.

2.3 Listado general de subastas

Metodo	GET
Endpoint	/api/auctions
Descripcion	Devuelve el listado general de subastas registradas.
Parametros	No requiere.
Body	No requiere.

Respuesta exitosa:


```
[
  {
    "id": 1,
    "fecha": "2026-06-05",
    "hora": "18:00",
    "estado": "abierta",
    "ubicacion": "Buenos Aires - Salon Principal",
    "categoria": "comun",
    "moneda": "pesos",
    "subastador": "Carlos Martillero"
  }
]
```


Errores posibles:

Codigo	Descripcion
500	Error interno al consultar subastas.

2.4 Subastas disponibles para un cliente

Metodo	GET
Endpoint	/api/clients/:clientId/auctions
Descripcion	Devuelve las subastas disponibles para un cliente, indicando si puede visualizar o pujar.
Parametros	clientId: identificador del cliente.
Body	No requiere.

Ejemplo:

```
GET /api/clients/1/auctions
```

Respuesta exitosa:

```
[
  {
    "id": 1,
    "fecha": "2026-06-05",
    "hora": "18:00",
    "estado": "abierta",
    "ubicacion": "Buenos Aires - Salon Principal",
    "categoria": "comun",
    "moneda": "pesos",
    "puedeVer": true,
    "puedePujar": true,
    "motivoBloqueo": null
  }
]
```

Reglas aplicadas:

- El usuario debe estar activo.
- El usuario debe estar admitido.
- La categoria del usuario debe ser igual o superior a la categoria de la subasta.
- El usuario debe tener al menos un medio de pago verificado para poder pujar.

Errores posibles:

Codigo	Descripcion
404	Cliente no encontrado.
500	Error interno del servidor.

2.5 Detalle de una subasta

Metodo	GET
Endpoint	/api/auctions/:auctionId
Descripcion	Devuelve los datos principales de una subasta.
Parametros	auctionId: identificador de la subasta.
Body	No requiere.

Ejemplo:

```
GET /api/auctions/1
```

Respuesta exitosa:


```
{
  "id": 1,
  "fecha": "2026-06-05",
  "hora": "18:00",
  "estado": "abierta",
  "ubicacion": "Buenos Aires - Salon Principal",
  "capacidadAsistentes": 100,
  "tieneDeposito": "si",
  "seguridadPropia": "si",
  "categoria": "comun",
  "moneda": "pesos",
  "subastador": "Carlos Martillero"
}
```


Errores posibles:

Codigo	Descripcion
404	Subasta no encontrada.
500	Error interno del servidor.

2.6 Catalogo de una subasta

Metodo	GET
Endpoint	/api/auctions/:auctionId/catalog
Descripcion	Devuelve los items del catalogo de una subasta.
Parametros	auctionId: identificador de la subasta.
Body	No requiere.

Ejemplo:

```
GET /api/auctions/1/catalog
```

Respuesta exitosa:


```
[
  {
    "itemId": 1,
    "productoId": 1,
    "descripcionCatalogo": "Reloj antiguo de coleccion",
    "descripcionCompleta": "Reloj antiguo de coleccion",
    "historia": "Pieza familiar en excelente estado de conservacion.",
    "precioBase": 10000,
    "comision": 1200,
    "subastado": "no",
    "vendido": "no",
    "mejorOferta": 12100
  }
]
```


Errores posibles:

Codigo	Descripcion
500	Error interno al consultar el catalogo.

2.7 Mejor oferta de un item

Metodo	GET
Endpoint	/api/catalog-items/:itemId/best-bid
Descripcion	Devuelve la mejor oferta actual de un item.
Parametros	itemId: identificador del item de catalogo.
Body	No requiere.

Ejemplo:

```
GET /api/catalog-items/1/best-bid
```

Respuesta exitosa:

```
{
  "itemId": 1,
  "precioBase": 10000,
  "moneda": "pesos",
  "mejorOferta": 12100
}
```

Errores posibles:

Codigo	Descripcion
404	Item no encontrado.
500	Error interno del servidor.

2.8 Registrar una puja

Metodo	POST
Endpoint	/api/bids
Descripcion	Registra una nueva puja para un item de una subasta.
Parametros	No requiere parametros por URL.
Body	Cliente, subasta, item e importe ofertado.

Body:

```
{
  "clienteId": 1,
  "subastaId": 1,
  "itemId": 1,
  "importe": 12200
}
```

Respuesta exitosa:

```
{
  "mensaje": "Puja registrada correctamente",
  "puja": {
    "identificador": 3,
    "importe": 12200,
    "fechaHora": "2026-05-16T20:35:00.000Z"
  }
}
```

Reglas aplicadas:

- El cliente debe existir.
- El cliente debe estar activo.
- El cliente debe estar admitido.
- El cliente debe tener al menos un medio de pago verificado.
- La subasta debe estar abierta o en curso.
- El item no debe estar vendido.
- La categoria del cliente debe permitirle pujar.
- La puja debe ser mayor que la mejor oferta actual.
- Para categorias distintas de oro y platino, minimo: mejor oferta + 1% del precio base, maximo: mejor oferta + 20% del precio base.

Errores posibles:

Codigo	Descripcion
400	Faltan datos o importe invalido.
403	Cliente no habilitado, sin medio de pago o categoria insuficiente.
404	Cliente, subasta o item no encontrado.
409	Item vendido o conflicto de estado.
500	Error interno del servidor.

2.9 Medios de pago del cliente

Metodo	GET
Endpoint	/api/clients/:clientId/payment-methods
Descripcion	Devuelve los medios de pago registrados del cliente.
Parametros	clientId: identificador del cliente.
Body	No requiere.

Ejemplo:

```
GET /api/clients/1/payment-methods
```

Respuesta exitosa:


```
[
  {
    "id": 1,
    "tipo": "tarjeta_credito",
    "entidad": "Visa",
    "numeroReferencia": "**** **** **** 1234",
    "esExtranjera": "no",
    "moneda": "pesos",
    "verificado": "si",
    "montoCheque": null,
    "montoDisponible": null
  }
]
```


Errores posibles:

Codigo	Descripcion
500	Error interno al consultar medios de pago.

2.10 Solicitud de articulo para subastar

Metodo	POST
Endpoint	/api/products
Descripcion	Permite que un usuario cargue un articulo propio para revision de la empresa.
Parametros	No requiere parametros por URL.
Body	Datos del articulo.

Body:

```
{
  "duenio": 1,
  "descripcionCatalogo": "Jarron antiguo",
  "descripcionCompleta": "Jarron de ceramica en buen estado",
  "historia": "Pertenecio a una coleccion familiar",
  "artistaDiseniador": "Sin identificar",
  "declaracionPropiedad": "si",
  "origenLicito": "si"
}
```

Respuesta exitosa:

```
{
  "mensaje": "Articulo enviado para revision",
  "producto": {
    "identificador": 2,
    "estadoAprobacion": "pendiente"
  }
}
```

Errores posibles:

Codigo	Descripcion
400	Faltan datos obligatorios o no se declaro propiedad.
500	Error interno del servidor.

2.11 Historial de participaciones

Metodo	GET
Endpoint	/api/clients/:clientId/history
Descripcion	Devuelve el historial de participaciones y pujas del cliente.
Parametros	clientId: identificador del cliente.
Body	No requiere.

Ejemplo:

```
GET /api/clients/1/history
```

Respuesta exitosa:


```
[
  {
    "subastaId": 1,
    "fecha": "2026-06-05",
    "hora": "18:00",
    "moneda": "pesos",
    "descripcionCatalogo": "Reloj antiguo de coleccion",
    "mejorOfertaPropia": 12200,
    "gano": 0
  }
]
```


Errores posibles:

Codigo	Descripcion
500	Error interno al consultar historial.

2.12 Perfil de usuario

Metodo	GET
Endpoint	/api/users/:userId
Descripcion	Devuelve los datos del usuario, estado, admision y categoria.
Parametros	userId: identificador del usuario.
Body	No requiere.

Ejemplo:

```
GET /api/users/1
```

Respuesta exitosa:

```
{
  "id": 1,
  "documento": "30123456",
  "nombre": "Alejandro",
  "apellido": "Salgado",
  "email": "alejandro@test.com",
  "telefono": null,
  "direccion": "Av. Siempre Viva 123",
  "estado": "activo",
  "admitido": "si",
  "categoria": "oro"
}
```

Errores posibles:

Codigo	Descripcion
404	Usuario no encontrado.
500	Error interno del servidor.

2.13 Notificaciones del cliente

Metodo	GET
Endpoint	/api/clients/:clientId/notifications
Descripcion	Devuelve las notificaciones privadas del cliente.
Parametros	clientId: identificador del cliente.
Body	No requiere.

Ejemplo:

```
GET /api/clients/1/notifications
```

Respuesta exitosa:


```
[
  {
    "id": 1,
    "titulo": "Bienvenido a Auct.io",
    "mensaje": "Tu usuario se encuentra habilitado para participar en subastas.",
    "fechaHora": "2026-05-16T20:00:00.000Z",
    "leida": "no"
  }
]
```


Errores posibles:

Codigo	Descripcion
500	Error interno al consultar notificaciones.

3. Manejo de errores

3.1 Errores de campos obligatorios

La app valida campos obligatorios antes de enviar informacion al backend.

Ejemplos:

- Login sin documento.
- Login sin clave.
- Solicitud de articulo sin descripcion.
- Solicitud de articulo sin aceptar declaracion de propiedad.
- Puja sin importe.

Mensajes mostrados al usuario:

- Ingresa el documento.
- Ingresa la clave.
- Ingresa una descripcion completa del articulo.
- Debes declarar que el bien te pertenece.
- Ingresa un importe para pujar.

3.2 Error de credenciales

Si el usuario ingresa documento o clave incorrecta, el backend responde con 401 - Documento o clave incorrecta. La app muestra el mensaje devuelto por la API.

3.3 Usuario no activo o no admitido

Si el usuario existe pero no esta activo o no fue admitido por la empresa, el backend responde con 403 - Usuario no activo o no admitido. La app no permite avanzar al Home.

3.4 Error de conexion

Si el backend no esta levantado, si la IP es incorrecta o si el telefono no puede conectarse a la PC, la app muestra: No se pudo conectar con el servidor. Este error se controla en los bloques catch de las actividades Android.

3.5 Usuario sin medio de pago verificado

Para poder pujar, el usuario debe tener al menos un medio de pago verificado. Si no lo tiene, el backend responde con 403 - El cliente no posee medios de pago verificados. La app muestra el motivo de bloqueo y no permite pujar.

3.6 Categoria insuficiente

Si la categoria del usuario es menor que la categoria requerida por la subasta, el sistema bloquea la puja. Ejemplo: usuario categoria comun intenta pujar en subasta oro. Respuesta esperada: 403 - La categoria del cliente no permite pujar en esta subasta.

3.7 Puja menor al minimo permitido

La puja debe superar la mejor oferta actual y respetar el minimo definido por la consigna: minimo = mejor oferta actual + 1% del precio base. Si el importe es menor, el backend responde con 400 - La puja minima permitida es ...

3.8 Puja mayor al maximo permitido

Para subastas que no son categoria oro o platino, la puja no puede superar: maximo = mejor oferta actual + 20% del precio base. Si el importe supera ese maximo, el backend responde con 400 - La puja maxima permitida es ...

3.9 Subasta cerrada o no disponible

Si la subasta no esta en estado abierta o en_curso, el backend no permite registrar una puja. Respuesta esperada: 403 - La subasta no esta abierta para recibir pujas.

3.10 Item vendido

Si el item ya fue vendido, el backend bloquea nuevas pujas. Respuesta esperada: 409 - El item ya fue vendido.

4. Circuito integrado probado

1. El usuario inicia sesion con documento y clave.
2. La app valida credenciales contra el backend.
3. El backend consulta SQL Server.

4. Si el login es correcto, la app guarda la sesion localmente.
5. El usuario ingresa al Home.
6. La app consulta subastas disponibles para el cliente.
7. El usuario abre el detalle de una subasta.
8. La app consulta el catalogo de esa subasta.
9. El usuario selecciona un item y realiza una puja.
10. El backend valida reglas de negocio.
11. El backend registra la puja en SQL Server.
12. La app muestra confirmacion.
13. El usuario puede consultar su historial y ver la participacion registrada.

5. Estado de avance

Para esta segunda entrega se considera implementado aproximadamente mas del 50% funcional del sistema, incluyendo:

- Backend conectado a SQL Server.
- API REST funcional.
- Frontend Android Java/XML.
- Login.
- Subastas disponibles.
- Detalle de subasta.
- Catalogo.
- Puja.
- Historial.
- Medios de pago.
- Solicitud de articulo.
- Perfil.
- Notificaciones.
- Splash.
- Cierre de sesion.
- Manejo basico de errores.

Quedan para una etapa posterior:

- Backend publicado online.
- App instalable final.
- Carga real de imagenes del producto.
- Tiempo real para actualizacion automatica de pujas.
- Cierre automatico de subasta.
- Determinacion formal del ganador.
- Registro completo de venta, pago, comision y envio.
- Gestion avanzada de multas.
- Visualizacion de polizas de seguro y ubicacion de piezas.